RESTAURANT MANAGEMENT SYSTEM

Full-Stack Web Application Project Documentation

React.js • Python • Django • Django REST Framework • JWT • SQLite

A complete technical and functional overview of the Restaurant Management System, including architecture, database, APIs, authentication, CRUD operations, filtering, pagination, dynamic menus, and external integrations.

Prepared by
Aatiqa Naaz

GitHub Repository
https://github.com/Aatiqa09/Restaurant-Management-System-

Table of Contents

1. Abstract

The Restaurant Management System is a full-stack web application developed to simplify the process of managing and exploring restaurant information. The application provides secure user authentication, restaurant creation, updating and deletion, search, category and availability filtering, pagination, restaurant details, ratings, restaurant-specific menu information, Google Maps integration, and external website navigation. The frontend is developed using React.js, while the backend is implemented using Python, Django, and Django REST Framework. Axios is used to communicate with REST APIs, JWT is used for authentication, and Django ORM manages the database. The result is a responsive and user-friendly platform for efficiently managing restaurant data.

2. Introduction

Managing restaurant information manually can become difficult as the amount of data increases. Details such as restaurant name, category, location, contact information, opening hours, menu items, delivery availability, and website information need to be organized and easily accessible.

The Restaurant Management System provides a centralized web platform where restaurant information can be created, viewed, updated, searched, filtered, and deleted. Users can also open a dedicated restaurant details page containing ratings, operating status, menu items, Google Maps location, and the restaurant's external website.

3. Problem Statement

Traditional or manually maintained restaurant information can make it difficult to efficiently search, update, organize, and access restaurant records. Users may also need to visit multiple platforms to find restaurant details, menus, locations, and official websites. The project addresses this problem by providing a centralized, interactive application backed by a REST API and database.

4. Objectives

- Develop a full-stack restaurant management application.
- Implement secure user authentication using JWT.
- Provide complete restaurant CRUD operations.
- Allow users to search and filter restaurants.
- Implement category, delivery, vegetarian, and non-vegetarian filtering.
- Implement pagination with five restaurants displayed per page.
- Display complete restaurant details and ratings.
- Display restaurant-specific menu items dynamically.
- Provide Google Maps and external website integration.
- Create a responsive and user-friendly interface.
- Demonstrate practical frontend-backend-database integration.

5. Scope of the Project

The system covers restaurant registration, authentication, restaurant CRUD operations, search, filtering, pagination, restaurant details, ratings, menu information, location integration, and external website access. The current application focuses on restaurant information management and can be extended later with ordering, reservations, payments, customer reviews, notifications, analytics, and restaurant-owner functionality.

6. Technologies Used

Technology	Purpose
React.js	Frontend interface and component-based UI
JavaScript	Frontend logic, events, state and API handling
HTML5	Page structure and semantic content
CSS3	Custom styling and responsive presentation
Bootstrap	UI components, layout and responsive styling
Python	Backend programming language
Django	Backend web framework and application structure
Django REST Framework	RESTful API development
JWT	Secure user authentication and protected requests
Axios	Communication between React and Django REST APIs
SQLite	Database used for development/storage in the project
Git	Version control
GitHub	Source-code hosting and project repository
VS Code	Development environment

Database note: The working project uses SQLite through Django's ORM and the development database file. MySQL can be used as an alternative relational database with the appropriate Django database configuration.

7. System Architecture

The application follows a client-server architecture. React.js is responsible for the frontend interface, Django REST Framework provides the API layer, Django handles backend logic and models, and the database stores persistent restaurant and menu data.

Layer	Responsibility
User	Interacts with the web application
React Frontend	Pages, components, forms, filters, navigation and UI state
Axios	Sends HTTP requests and receives JSON responses
Django REST API	Handles CRUD operations and exposes application data
JWT Authentication	Validates authenticated requests
Django ORM	Maps Python models to database tables
Database	Stores users, restaurants and menu information

```
USER → REACT FRONTEND → AXIOS → DJANGO REST API → DJANGO ORM → DATABASE
```

```
DATABASE → DJANGO REST API → JSON RESPONSE → REACT FRONTEND → USER
```

8. Application Workflow

- User opens the application.
- New users can register; existing users can log in.
- Django validates credentials and issues JWT tokens.
- The authenticated user accesses the dashboard and restaurant management features.
- Restaurant data is retrieved through REST APIs and rendered by React.
- Users can search, filter and paginate restaurant results.
- Users can add, edit or delete restaurants.
- Selecting View Details opens the restaurant details page.
- Restaurant-specific menu data is retrieved dynamically.
- Google Maps and external website buttons open the relevant external resources.
- Database changes are reflected dynamically in the frontend.

9. Database Design

The application uses Django models and Django ORM to represent restaurant and menu information. A restaurant can have multiple menu items, creating a one-to-many relationship between Restaurant and MenuItem.

Entity	Relationship	Description
User	Authentication	Stores user credentials/account information
Restaurant	1-to-many	One restaurant can contain multiple menu items
MenuItem	many-to-1	Each menu item belongs to one restaurant

Restaurant Model – Main Fields

Field	Description
id	Unique restaurant identifier
name	Restaurant name
description	Restaurant description
category	Restaurant category
address	Restaurant location
phone_number	Contact phone number
email	Restaurant email
opening_time	Restaurant opening time
closing_time	Restaurant closing time
price_range	Restaurant price category
is_vegetarian_friendly	Indicates vegetarian-friendly availability
has_delivery	Indicates delivery availability
website	External restaurant website URL

image_url	Online restaurant image URL

MenuItem Model – Main Fields

Field	Description
id	Unique menu item identifier
restaurant	Reference to the associated restaurant
name	Menu item name
description	Menu item description
price	Menu item price
rating	Menu item rating where applicable

Relationship

```
Restaurant 1 ■■■■■■■■■ * MenuItem
```

This relationship ensures that the details page for a particular restaurant displays only the menu items associated with that restaurant.

10. Backend Development

The backend is developed with Python, Django and Django REST Framework. Django provides the application structure, model layer, routing and administrative interface, while Django REST Framework exposes the application functionality through REST APIs.

Backend Component	Role
models.py	Defines Restaurant and MenuItem data structures
serializers.py	Validates input and converts model data to/from JSON
views.py / ViewSets	Handles REST API operations such as list, create, update and delete
urls.py	Maps API URLs to backend views
admin.py	Provides Django Admin management interface
signals.py	Supports backend event-driven logic where configured
Django ORM	Connects Python models with the database

CRUD Operations

CRUD stands for Create, Read, Update and Delete. The project implements these operations for restaurant management through REST APIs.

Operation	Example	Result
Create	Add Restaurant	Creates a new restaurant record
Read	Restaurant List / Details	Retrieves restaurant information
Update	Edit Restaurant	Updates existing restaurant information
Delete	Delete Restaurant	Removes the selected restaurant

11. REST API Documentation

Method	Endpoint	Purpose
POST	/api/users/register/	Register a new user
POST	/api/token/	Authenticate user and obtain JWT tokens
POST	/api/token/refresh/	Refresh an access token
GET	/api/restaurants/	Retrieve restaurant records
POST	/api/restaurants/	Create a restaurant
GET	/api/restaurants/{id}/	Retrieve one restaurant
PUT/PATCH	/api/restaurants/{id}/	Update restaurant details
DELETE	/api/restaurants/{id}/	Delete a restaurant

The frontend communicates with these APIs using Axios. Requests and responses are exchanged as JSON data. The exact MenuItem endpoint can be documented from the project's current Django URL configuration if a separate menu route is exposed.

12. Authentication & Security

The project uses JWT (JSON Web Token) authentication. During login, the backend validates the submitted credentials and returns authentication tokens. The frontend uses the access token for authenticated API requests. Protected frontend routes prevent unauthorized access to restricted application pages.

Step	Process
1	User enters login credentials
2	React sends credentials to Django's token endpoint
3	Django validates the credentials
4	JWT access and refresh tokens are generated
5	Frontend stores the required token information
6	Axios includes the access token in authenticated requests
7	Backend validates the token before protected operations

13. Frontend Development

The frontend is developed using React.js. The application is organized into reusable components and pages. React Router handles navigation, React hooks manage state and side effects, and Axios communicates with the backend APIs.

Frontend Area	Purpose
Login	User authentication
Register	New account creation
Dashboard	Quick access and restaurant statistics
RestaurantList	Search, filters, pagination and restaurant cards
RestaurantDetails	Complete restaurant information and menu
AddRestaurant	Restaurant creation form
Edit Restaurant	Restaurant update form
RestaurantCard	Reusable restaurant display component
ProtectedRoute	Restricts protected pages to authenticated users
Layout / Navbar	Common navigation and page structure
services/api.js	Centralized Axios/API communication

Frontend–Backend Communication

```
React Component → Axios → REST API → Django ViewSet → Serializer → Model/ORM → Database
Database → Model/Serializer → JSON Response → Axios → React State → UI
```

14. Application Features

14.1 User Authentication

- Login page accepts username/email and password.
- Django verifies credentials and generates JWT authentication tokens.
- Forgot Password provides an administrator-contact message for password reset.
- Users without an account can navigate to the registration page.
- Registration checks whether the requested account already exists before creating a new user.
- Successful registration displays a confirmation message and allows the user to log in.

14.2 Dashboard

- Provides quick navigation to restaurant management features.
- Displays restaurant statistics.
- Provides access to the Restaurants section and Add Restaurant functionality.
- Provides secure Logout functionality.

14.3 Restaurant Listing, Search & Filtering

- Displays Total Restaurants, Delivery Available, Vegetarian and Non-Vegetarian statistics.
- Clicking a statistic filters the restaurant list according to that criterion.
- Category filters include Indian, Italian, Fast Food and Café.
- Multiple filters can be combined; for example, Delivery + Fast Food displays only matching restaurants.
- Search dynamically filters restaurants based on the search text.
- Pagination displays five restaurants per page.
- Each restaurant card provides Edit, Delete and View Details actions.

14.4 Restaurant Management

- Add Restaurant creates a new restaurant record.
- Edit Restaurant loads existing information into a dedicated form.
- Update Restaurant saves modified information.
- Delete Restaurant removes the selected restaurant.
- Success messages provide feedback after create, update and delete operations.
- Dashboard statistics update dynamically after restaurant data changes.

14.5 Restaurant Details

- Displays restaurant image, name, rating and reviews.
- Displays current Open Now / Closed status based on operating hours.
- Displays category, address, contact information, website, opening hours and price.
- Provides Google Maps and Visit Website actions.
- Displays the restaurant-specific menu dynamically.

14.6 Dynamic Menu

- Menu items are retrieved from the backend rather than being hard-coded into the page.
- Each menu item displays its name, description and price.
- Menu data is associated with the selected restaurant.
- Opening a restaurant therefore displays only that restaurant's related menu items.

15. Screenshots & Feature Demonstration

15.1 Restaurant Listing Page

The Restaurant Listing page provides a complete restaurant discovery interface. The dashboard statistics show the current restaurant totals and provide quick filters. Category buttons, search, pagination, and restaurant-card actions make it possible to manage and locate restaurant information efficiently.

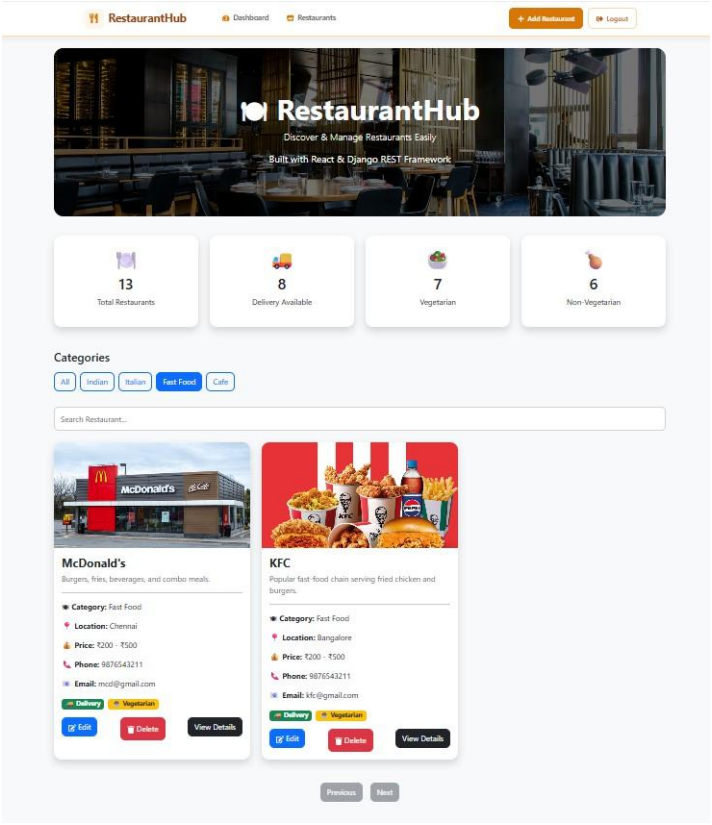

Figure 1: Restaurant Listing Page with statistics, category filters, search and pagination

Filtering example: selecting Delivery and Fast Food together narrows the list to restaurants satisfying both conditions. In the current dataset, this combination displays two restaurants.

Pagination: the interface displays five restaurants per page and provides Previous and Next navigation.

15.2 Add Restaurant

Clicking Add Restaurant opens a dedicated form where users can enter restaurant name, description, category, price range, address, phone number, email, website, image URL, opening and closing hours, vegetarian-friendly status, and home-delivery availability.

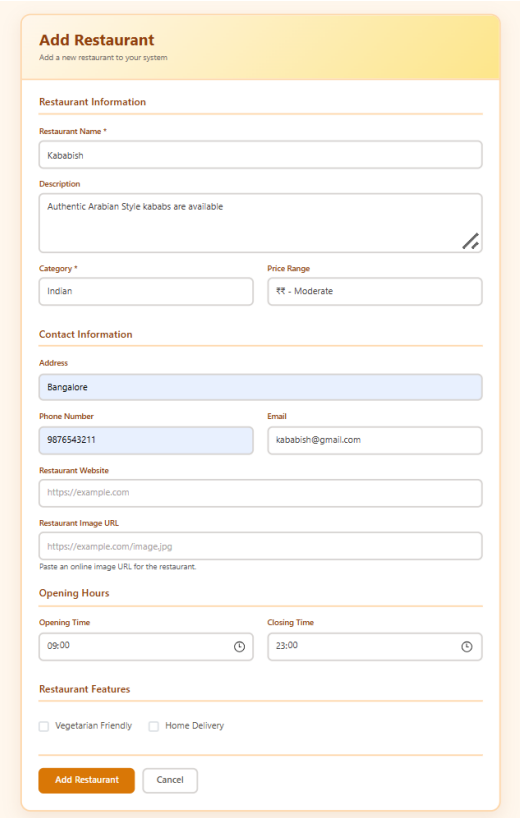

Figure 2: Add Restaurant form

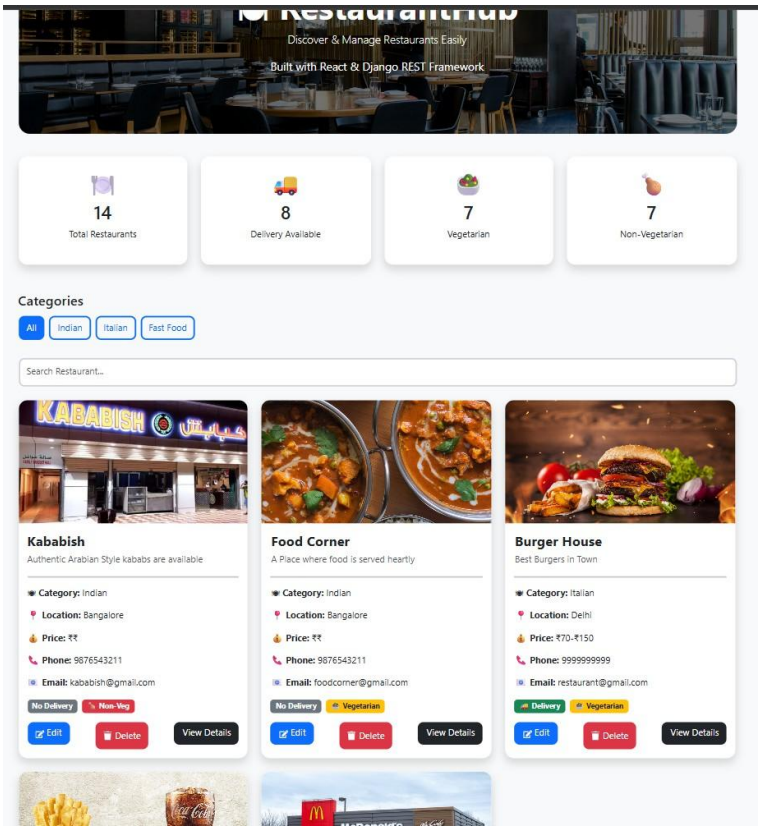

Figure 3: Restaurant list after successful addition; the total count is updated dynamically

After submission, the application displays a success message and the newly created restaurant becomes available in the restaurant list. The restaurant count is recalculated from the latest backend data.

15.3 Edit Restaurant

Clicking Edit on a restaurant card navigates to the Edit Restaurant page. Existing values are loaded into the form so the user can modify restaurant information. Clicking Update Restaurant sends the changes to the backend and displays a success confirmation.

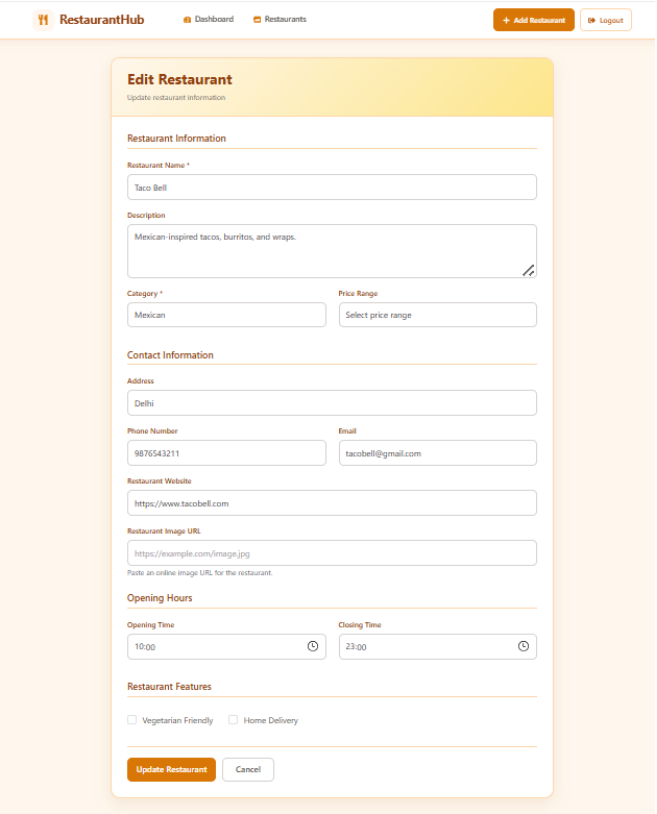

Figure 4: Edit Restaurant page with pre-filled restaurant information

15.4 Delete Restaurant

Each restaurant card includes a Delete button. The selected restaurant is removed through the backend API, after which the application displays a confirmation message such as “Restaurant deleted”. The restaurant list and related statistics are refreshed.

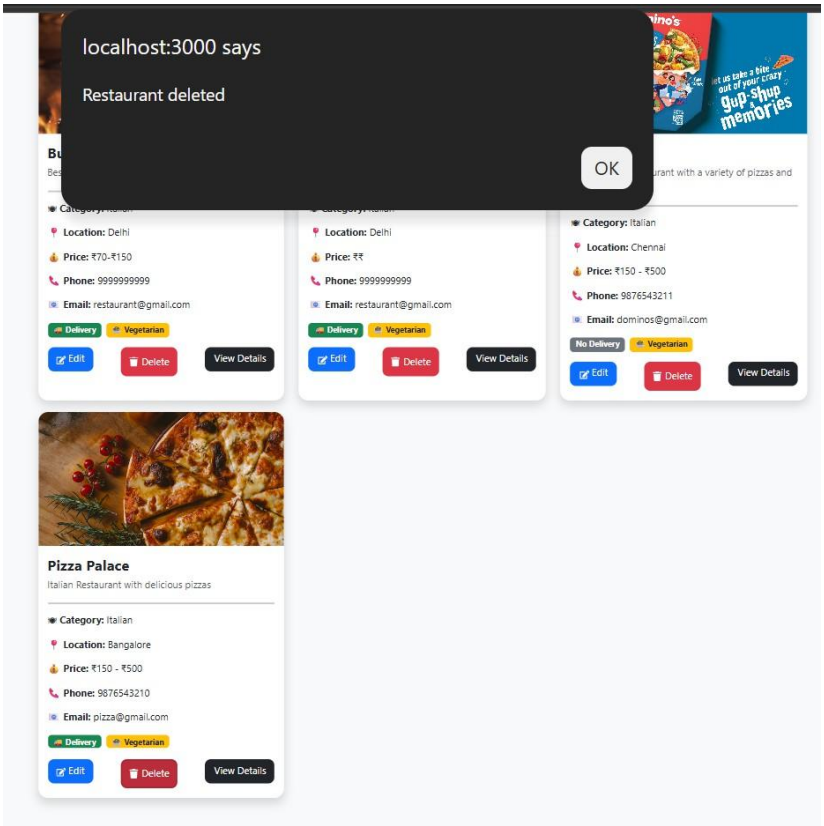

Figure 5: Delete operation with successful confirmation message

15.5 Restaurant Details Page

The View Details button opens a dedicated page containing the restaurant's complete information, rating, current operating status, contact information, opening hours, price range, external links and menu.

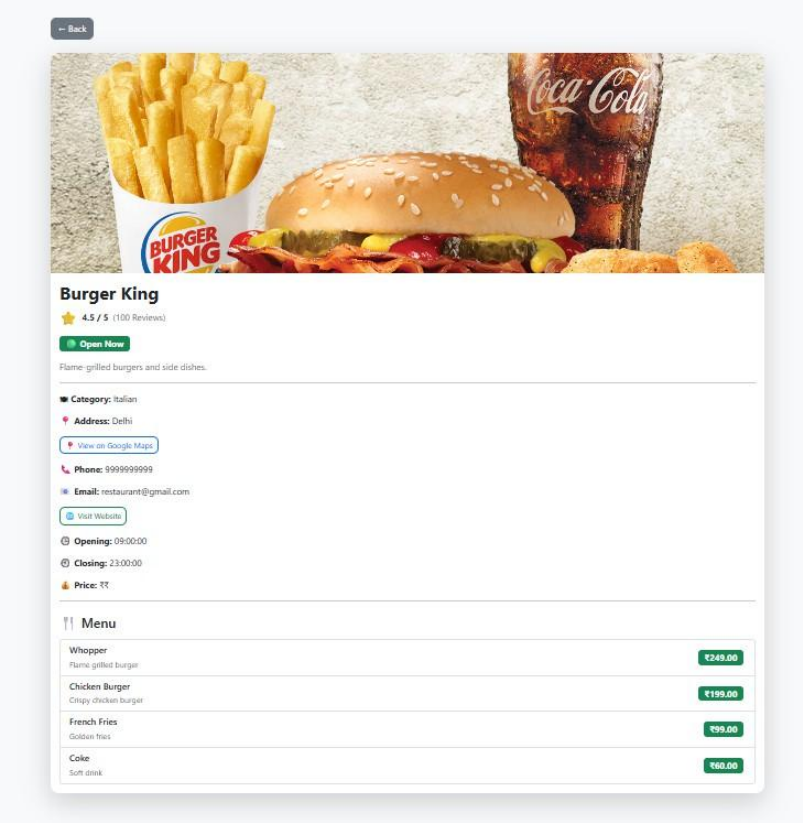

Figure 6: Restaurant Details page

15.6 Dynamic Restaurant Menu

The menu section is populated dynamically for the selected restaurant. In the example below, Kababish has its own menu containing Special Veg Platter, Paneer Tikka, Butter Naan and Fresh Lime Soda with their descriptions and prices.

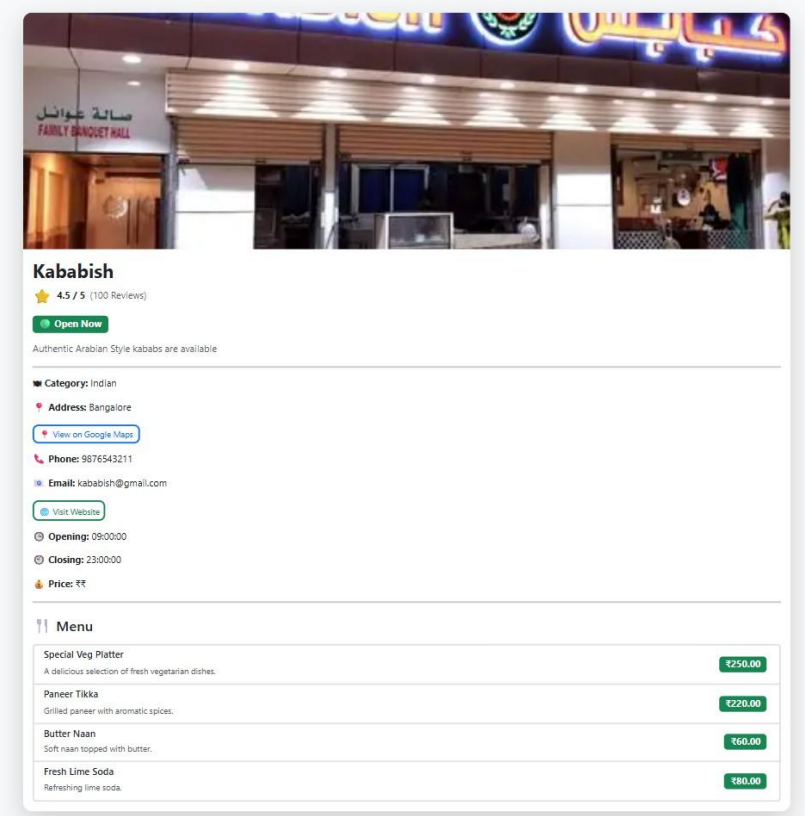

Figure 7: Restaurant-specific dynamic menu

15.7 External Website Integration

The Visit Website button uses the stored restaurant website URL and opens the external restaurant website, allowing the user to access additional information, offers, menus or services.

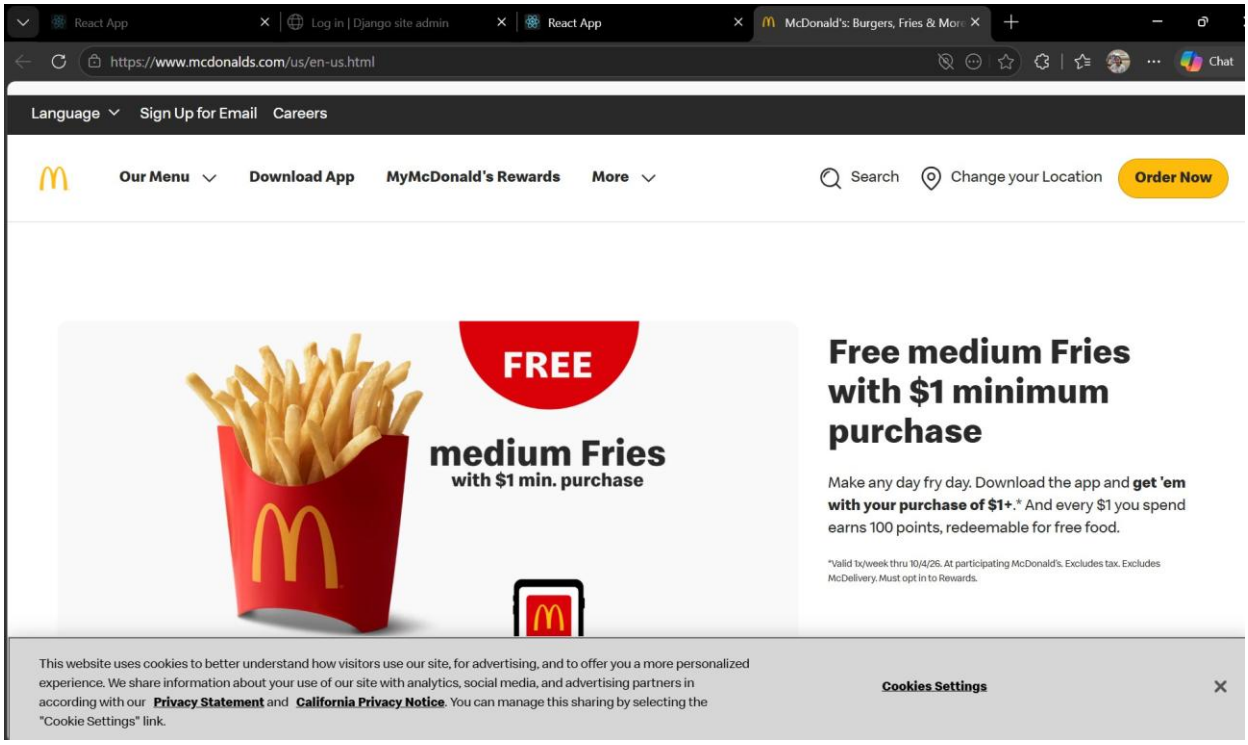

Figure 8: External restaurant website opened from the application

15.8 Google Maps Integration

The View on Google Maps button opens Google Maps using the restaurant's address/location, allowing users to view the location and obtain directions.

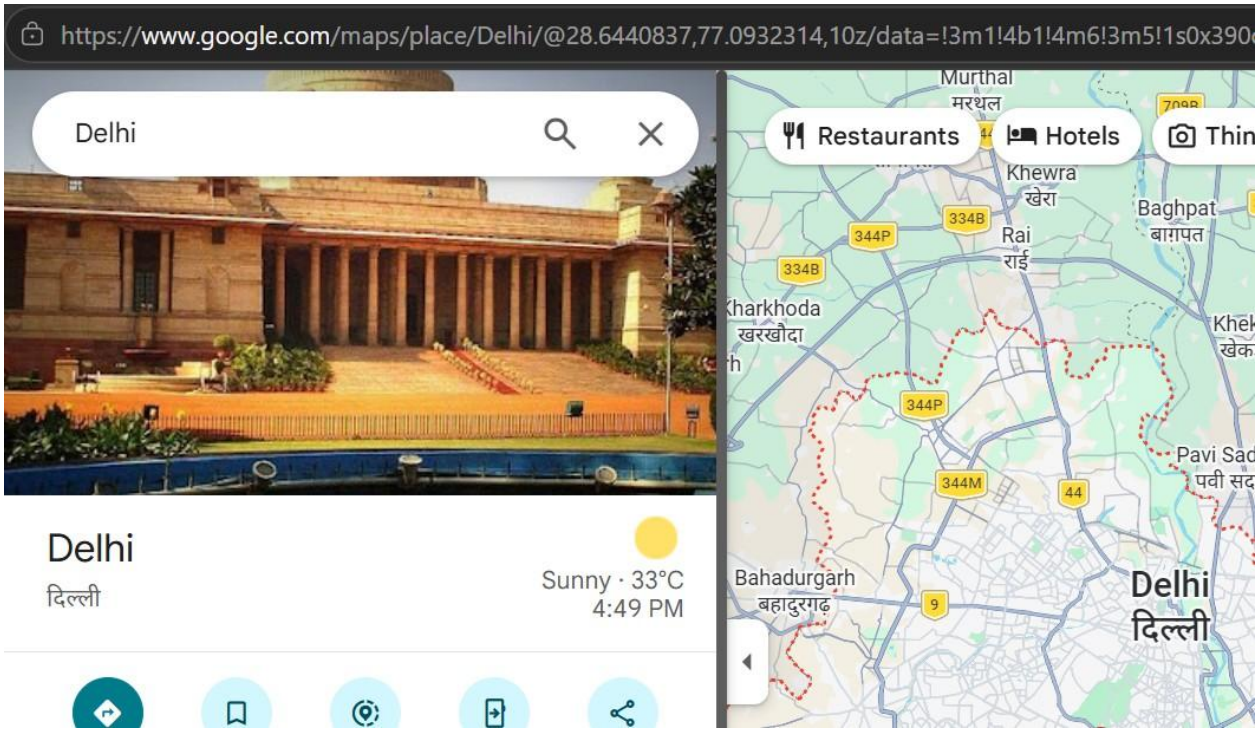

Figure 9: Google Maps opened from the restaurant details page

16. Django Administration & Administrative Access

The project also includes Django's built-in administration interface as a backend management layer. Authorized administrators can manage application data directly from the Django Admin site. This complements the React frontend by providing centralized control over users, authentication data, restaurants, and menu items.

Admin Authentication

The Django Admin interface has its own administrative login. Only an authorized administrator can access the backend management area. After successful login, the administrator can access the available management sections and perform administrative operations.

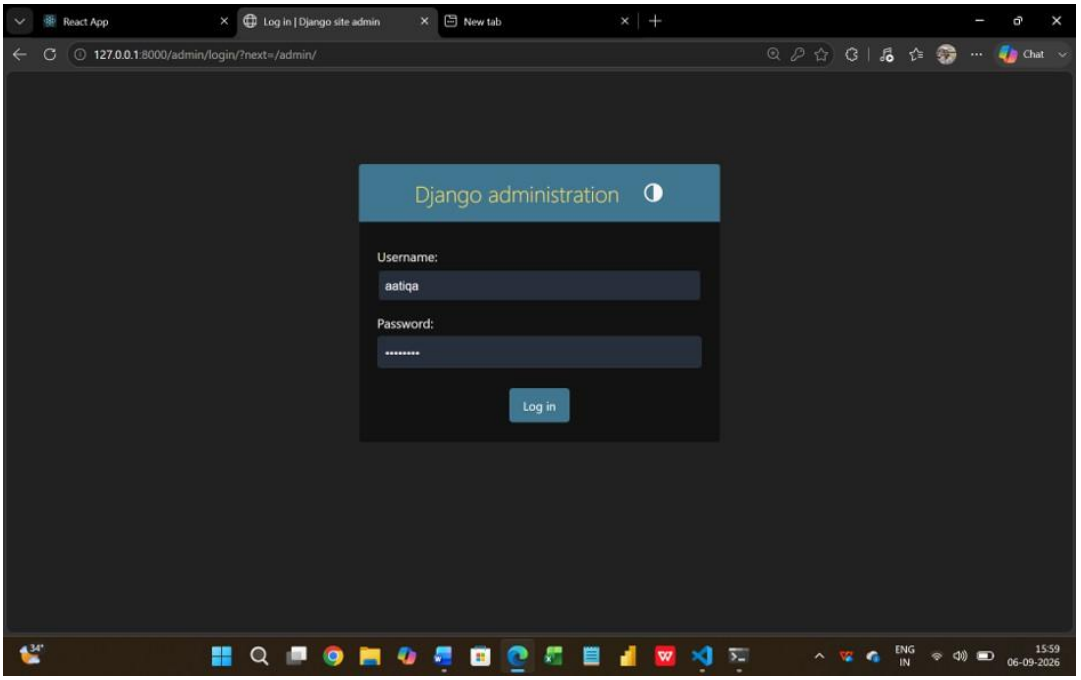

Figure 10: Django Administration Login

Why the Admin panel is important: It provides a secure backend interface for maintaining the application's database records without depending only on the frontend. Changes made through the admin interface are reflected in the application data.

16.1 Django Admin Dashboard

After logging in, the administrator is presented with the Django Admin dashboard. The dashboard provides access to authentication and authorization data as well as the Restaurant and Menu Item models used by the project.

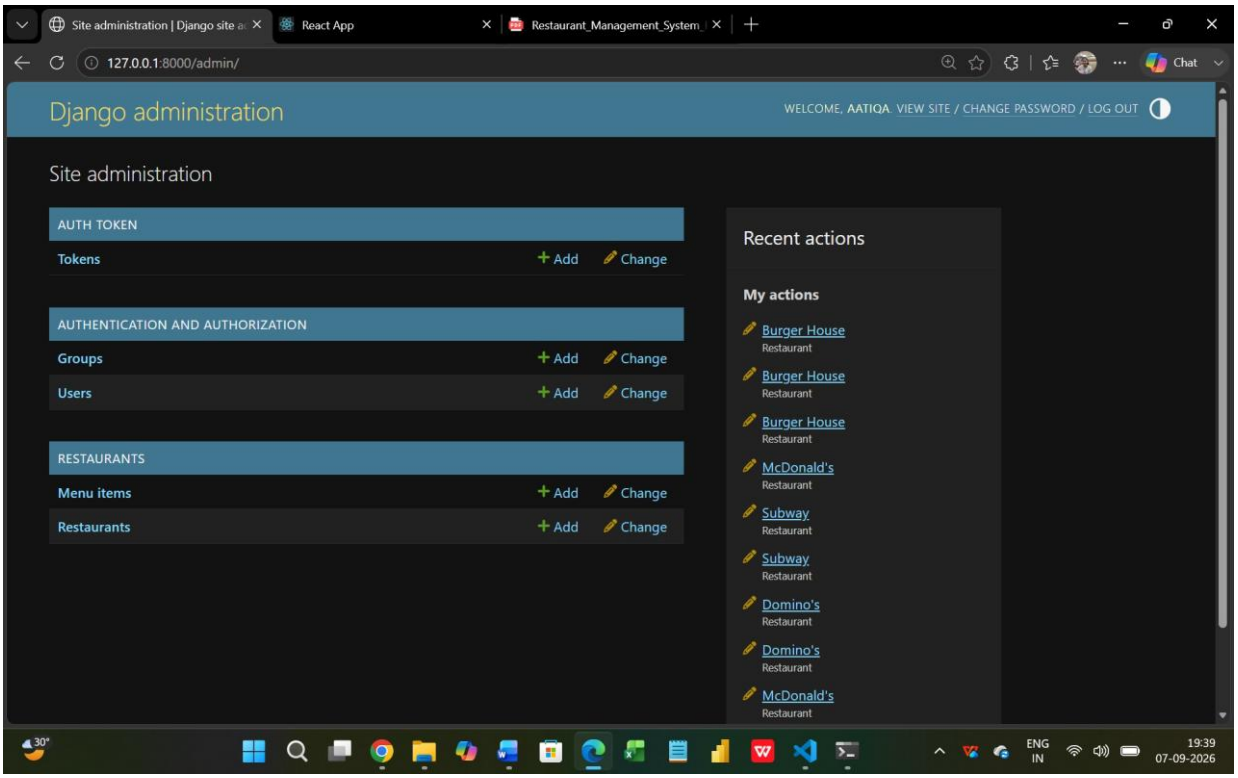

Figure 11: Django Admin Dashboard

Administrative areas visible in the project:

- **Users:** View and manage registered user accounts.
- **Groups:** Manage Django authorization groups where configured.
- **Tokens:** Access authentication-token management provided by the project configuration.
- **Restaurants:** Add, view, modify, and remove restaurant records.
- **Menu Items:** Add, view, modify, and remove menu records linked to restaurants.

16.2 User Management

The Users section allows the administrator to view registered accounts from the backend. The admin can inspect user records and use Django's account-management controls to maintain user access.

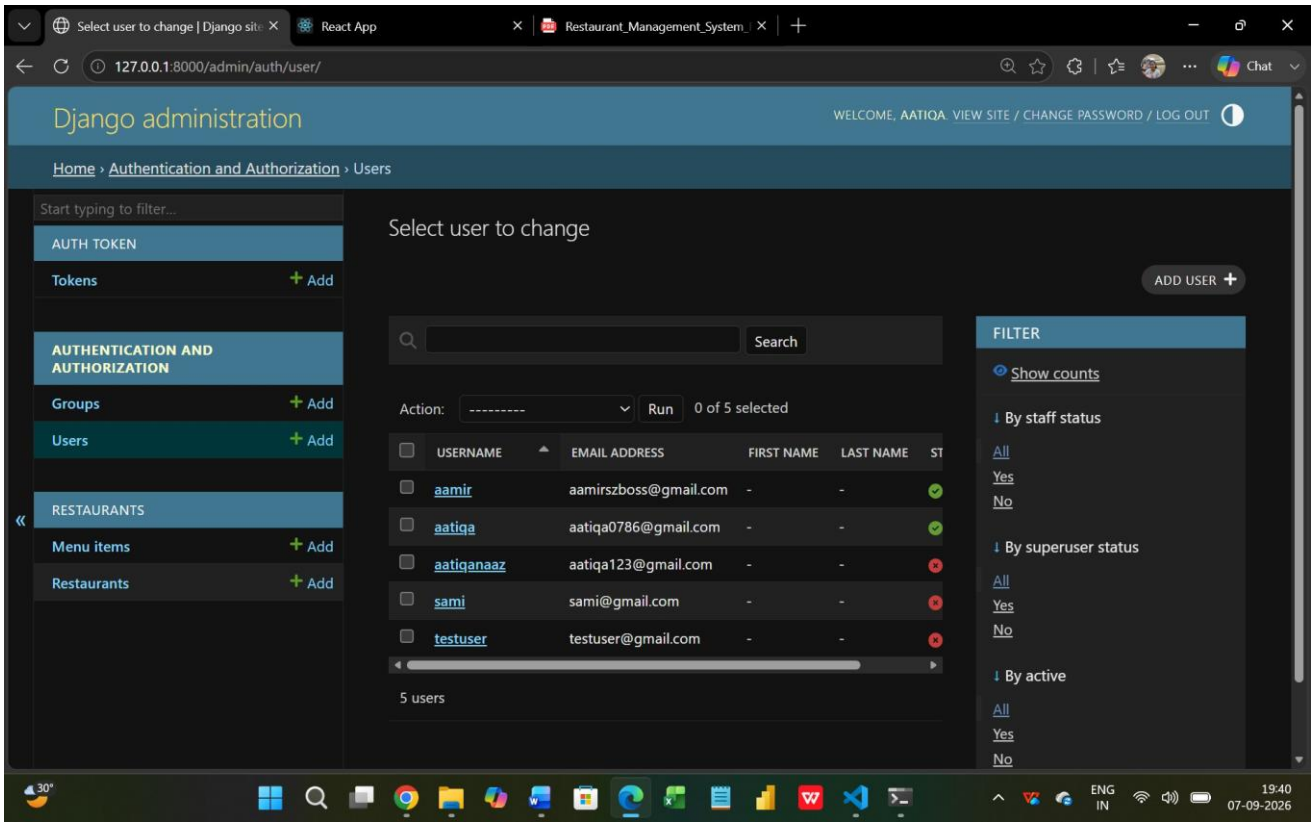

Figure 12: Django Admin – User Management

Administrator capabilities for users:

- View existing registered user accounts.
- Add new users from the Django Admin interface.
- Update user information and account settings.
- Change the password of an existing user through Django's user-management interface.
- Manage staff/superuser-related permissions and account status where applicable.

This gives the administrator control over user accounts in addition to the JWT-based login flow available in the React application.

16.3 Restaurant Management through Django Admin

The Restaurants section provides direct backend access to the restaurant records stored in the database. The administrator can maintain restaurant information independently of the frontend forms.

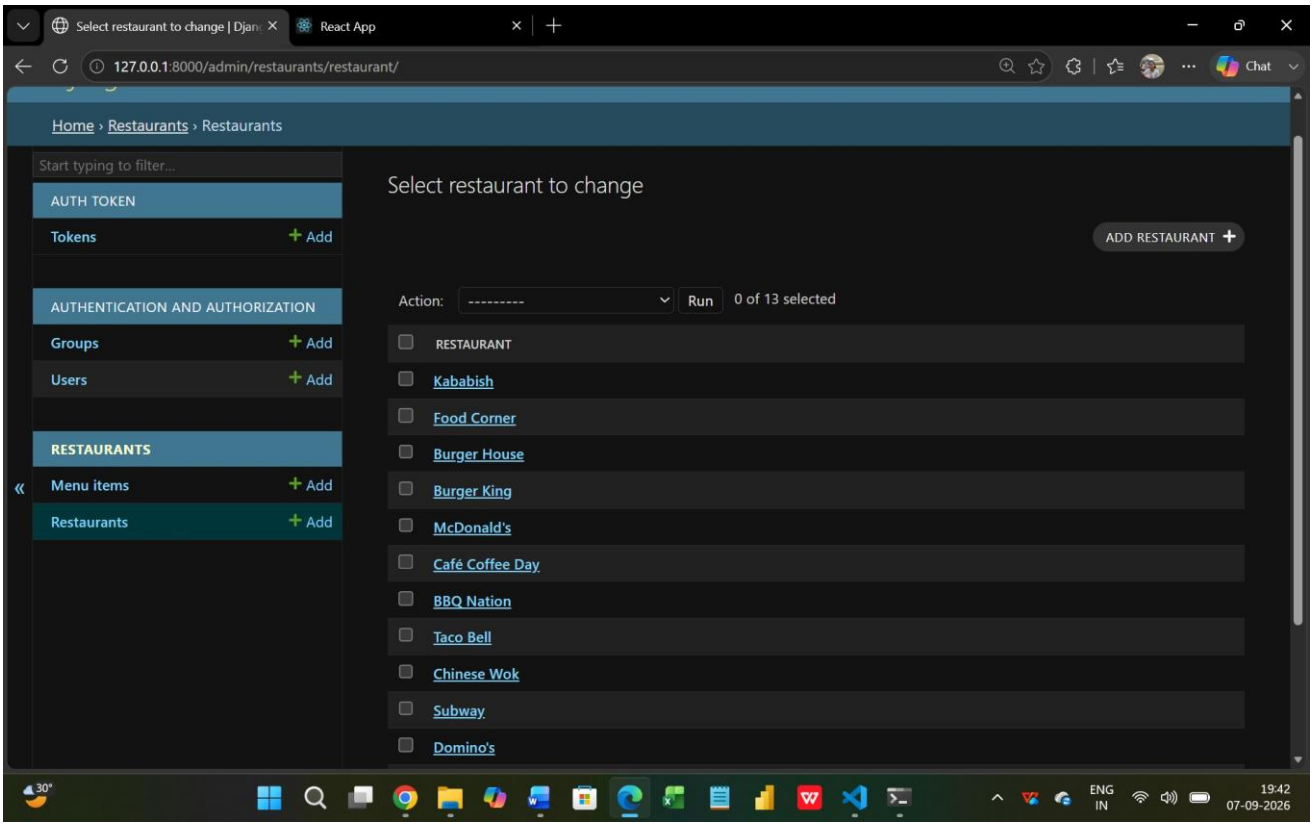

Figure 13: Django Admin – Restaurant Management

Restaurant administration operations:

- Add a new restaurant record.
- View and search existing restaurant records.
- Update restaurant information such as name, category, address, contact details, price range, website, image URL, opening hours, and features.
- Delete an entire restaurant record when it is no longer required.

Because the admin interface operates on the Django models, administrative changes are stored in the same database used by the REST API and frontend.

16.4 Menu Item Management & Complete Admin Control

The Menu Items section allows the administrator to manage menu records and their association with restaurants. This supports the dynamic menu feature shown on each restaurant's details page.

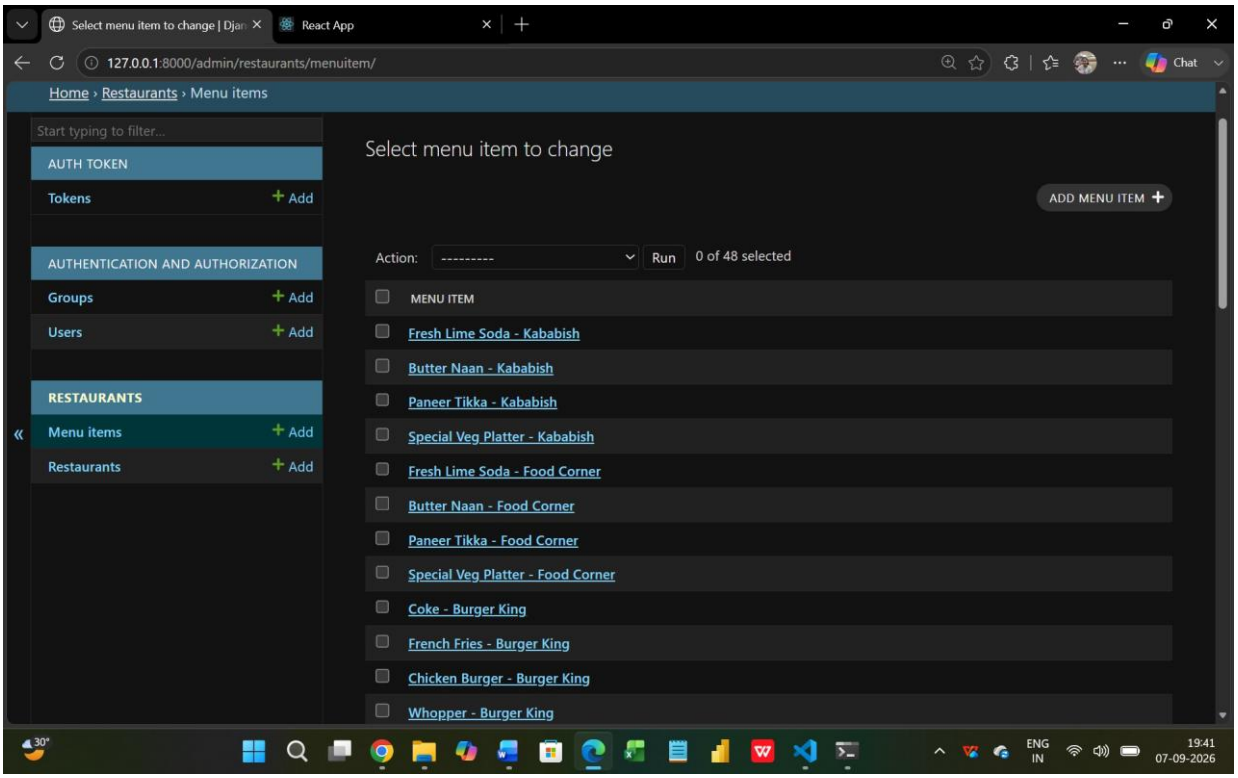

Figure 14: Django Admin – Menu Item Management

Menu administration operations:

- Add new menu items and associate them with a restaurant.
- Update menu item names, descriptions, prices, and other available fields.
- Remove menu items from the system.
- Maintain restaurant-specific menus that are displayed dynamically on the frontend.

Overall Administrative Access

The Django Admin acts as a centralized administrative control panel. Depending on the permissions of the logged-in administrator, the admin can add, remove, and update restaurant records, menu items, and user accounts. The administrator can also change an existing user's password and manage other available Django authentication/authorization records. This provides broad backend control over the application's stored data.

Frontend vs. Admin Management: The React application provides the user-facing restaurant management workflow, while Django Admin provides direct backend/database administration. Both interact with the same Django models and database.

17. Testing

Functional testing was used to verify that the major user flows work correctly. The following test cases cover authentication, restaurant management, search/filtering, details, menus and integrations.

Test Case	Action	Expected Result	Status
TC01	Register with valid details	Account is created	Pass
TC02	Register existing account	User is notified	Pass
TC03	Login with valid credentials	JWT authentication succeeds	Pass
TC04	Login with invalid credentials	Login is rejected	Pass
TC05	Add restaurant	Restaurant is created and displayed	Pass
TC06	Edit restaurant	Restaurant information is updated	Pass
TC07	Delete restaurant	Restaurant is removed	Pass
TC08	Search restaurant	Matching results displayed	Pass
TC09	Apply category filter	Matching category displayed	Pass
TC10	Apply combined filters	Only matching results displayed	Pass
TC11	Use pagination	Five restaurants shown per page	Pass
TC12	Open restaurant details	Details page displayed	Pass
TC13	View dynamic menu	Related menu items displayed	Pass
TC14	Open Google Maps	Location opens in Google Maps	Pass
TC15	Visit restaurant website	External website opens	Pass

18. Challenges & Solutions

Challenge	Solution
JWT/API authorization issues	Configured authentication and appropriate permission handling for protected operations.
Frontend-backend communication	Used Axios with REST API endpoints and authorization headers.
Repeated restaurant images	Used individual image URLs with category/fallback image handling.
Dynamic restaurant statistics	Refreshed restaurant data and recalculated totals after data changes.
Search and multiple filters	Implemented state-based filtering and combined conditions for restaurant discovery.
Large restaurant list	Implemented pagination with five restaurants per page.
Restaurant-specific menus	Linked MenuItem records to restaurants and retrieved related data dynamically.

19. Future Enhancements

- Online food ordering and shopping cart functionality.
- Table reservation and booking system.
- Online payment integration.
- Customer reviews and comments.
- Advanced restaurant and menu-item rating system.
- Restaurant-owner accounts and role-based access.
- Admin analytics and reporting dashboard.
- Email/SMS notifications.
- Advanced search, sorting and distance-based discovery.
- Image upload instead of only external image URLs.
- Cloud deployment and production database configuration.
- Order history, favourites and personalized recommendations.

20. Conclusion

The Restaurant Management System successfully demonstrates the development of a full-stack web application using React.js, Python, Django, Django REST Framework, JWT authentication and a relational database. The system provides practical restaurant management features including secure authentication, CRUD operations, search, filtering, pagination, restaurant details, ratings, dynamic menus, Google Maps integration and external website navigation.

The project also demonstrates how a modern frontend communicates with a RESTful backend and database. Through this implementation, important concepts such as React components, React Router, Axios, REST APIs, Django ORM, serializers, JWT authentication, CRUD operations, database relationships and dynamic UI updates have been applied in a practical project.

21. GitHub Repository

The complete source code and project implementation are available on GitHub:

Restaurant Management System
https://github.com/Aatiqa09/Restaurant-Management-System-

The repository can be used to review the frontend, backend, API implementation, database configuration and project structure.